

Plano de Engenharia de Software, Segurança e Infraestrutura V4: fm_blackmesa (www.blackmesa.page)

Este documento consolida o plano mestre de arquitetura, segurança (OWASP Top 10) e infraestrutura para o portal **www.blackmesa.page** e sua estação de rádio associada, a **fm_blackmesa**. O ecossistema é projetado sob a premissa de custo zero de hospedagem (Free-Tier), descentralização de permissões sem autenticação tradicional (Passwordless / Signed URLs) e automação completa do ciclo de vida dos conteúdos.

1. Matriz Unificada de Arquitetura e Distribuição de Infraestrutura

Tecnologia	Ambiente de Hospedagem	Papel no Ecossistema	Vantagem de Segurança / Custo
SvelteKit (Frontend)	Cloudflare Pages	Interface pública da rádio, formulários de submissão e painel de moderação.	Custo Zero. Proteção DDoS nativa de borda da Cloudflare e entrega global ultrarrápida.
Rust Axum (Backend)	Oracle Cloud (Always Free)	Gateway de API, validação binária de mídia, proxy Spotify e gerenciamento de sessões efêmeras.	Execução nativa de alta performance em arquitetura ARM Ampere (até 4 vCPUs / 24GB RAM inclusos).
Liquidsoap & Icecast	Oracle Cloud (Always Free)	Motor de áudio, fallback dinâmico e servidor de streaming (transmissor).	Processamento de áudio em tempo real sem custo com isolamento de portas internas da VPS.

Tecnologia	Ambiente de Hospedagem	Papel no Ecosistema	Vantagem de Segurança / Custo
Supabase	Supabase Free Tier	Persistência relacional, Row Level Security (RLS) e barramento de webhooks reativos.	Isolamento robusto multi-tenant nativo via Postgres sem necessidade de gerenciar infra de banco.

2. Plano 1: Motor de Áudio, Máquina de Estados e Metadados Real-time (Liquidsoap + SSE)

O motor de áudio opera continuamente na VPS da Oracle Cloud através de uma hierarquia rígida de fontes não-bloqueantes (``track_sensitive=false``). A sincronia de metadados ("Now Playing") elimina o polling no cliente: eventos disparados pelo Liquidsoap notificam o backend Rust que repassa o estado via Server-Sent Events (SSE) para os ouvintes do portal.

Modelo de Meta-Prompt para a IA Arquiteta (Plano 1)

Aja como um Arquiteto de Sistemas de Streaming de Áudio. Desenvolva um plano técnico detalhado para que outra IA escreva:

- Um arquivo de configuração do Liquidsoap v2.x contendo três fallbacks: ``input.harbor`` para conexões de DJs ao vivo, um ``request.dynamic`` alimentado por chamadas HTTP e uma playlist local de failover.
 - A lógica para interceptar metadados através da função ``on_track`` do Liquidsoap e enviá-los via POST HTTP para uma API externa.
 - Um endpoint em Rust (framework Axum) que monte um barramento SSE (``/api/stream/now-playing``) para propagar de forma reativa os metadados para os clientes.
- Instrua a IA de programação a incluir tratamento robusto de desconexão e garantia de não-bloqueio de threads de áudio.

3. Plano 2: Persistência de Dados e Controle de Acesso Seguro (Supabase RLS)

A segurança ao nível de banco de dados isola rigidamente os registros das tabelas, assegurando que usuários públicos, artistas temporários e administradores possuam

escopos estritamente delimitados, mitigando falhas do OWASP A01 (Broken Access Control).

Modelo de Meta-Prompt para a IA Arquiteta (Plano 2)

Aja como um Administrador de Banco de Dados especialista em PostgreSQL e Supabase Security. Desenvolva as especificações para que outra IA gere o script de migração completo:

1. Definição das tabelas: `perfis`, `slots\_grade`, `inscricoes\_open\_call` e `curadorias` (com suporte a enums nativos de controle de status).
 2. Habilitação de Row Level Security (RLS) em todas as tabelas.
 3. Criação de políticas RLS explícitas: usuários públicos só inserem em inscrições e curadorias e leem curadorias apenas se aprovadas. Artistas só modificam seus próprios slots de áudio se o horário atual for inferior ao horário de início do programa gravado.
- Exija que a IA final apresente as queries limpas e otimizadas com indexação em chaves estrangeiras.

4. Plano 3: Autenticação Criptográfica por Rotas Assinadas (Signed URLs / No-Login)

O sistema da fm_blackmesa opera sem formulários de login tradicionais. O acesso a painéis de upload ou administração é feito por URLs assinadas de uso único geradas pelo servidor Rust com payloads JWT criptografados e Time-To-Live (TTL) curto, persistidos em cookies de segurança estrita.

Modelo de Meta-Prompt para a IA Arquiteta (Plano 3)

Aja como um Engenheiro de Criptografia de Software Aplica. Crie uma especificação para orientar uma IA a programar um sistema de autenticação sem senha (passwordless) em Rust:

1. O backend deve gerar tokens JWT contendo claims específicas (permissões, ID do recurso associado e timestamp de expiração rígido de no máximo 48 horas).
2. Rotas protegidas devem extrair o token contido nos parâmetros da URL, validá-lo e assentá-lo em um cookie de sessão assinado configurado como HttpOnly, Secure e SameSite=Strict.
3. Um mecanismo de revogação baseado em blacklist ou verificação de chaves no banco Supabase para invalidar instâncias vazadas antes do prazo padrão de expiração.

5. Plano 4: Ingestão de Mídia, Sanitização de Arquivos e Coleta de Lixo (Garbage Collection)

A esteira de arquivos previne ataques de injeção (OWASP A03) e exaustão de armazenamento de borda. O backend analisa assinaturas binárias (Magic Bytes), reescreve metadados e nomes através de UUIDv4 usando processos controlados de

FFmpeg, e executa uma rotina de Cron Job em segundo plano para purgar mídias reproduzidas.

Modelo de Meta-Prompt para a IA Arquiteta (Plano 4)

Aja como um Engenheiro de Confiabilidade de Sistemas (SRE) e Especialista em AppSec. Desenvolva um roteiro completo para guiar uma IA na criação de uma esteira segura de arquivos em Rust:

1. Uma rota de upload multipart em Axum limitada a 200MB que leia o cabeçalho binário (Magic Bytes) usando a biblioteca ``infer`` para bloquear falsificações de extensão.
2. Renomeação mandatória dos arquivos salvos em formato UUIDv4, descartando totalmente a string original para anular Path Traversal.
3. Uso seguro do módulo ``std::process::Command`` para invocar o binário do ``ffmpeg`` com argumentos sanitizados para limpar tags ID3 e padronizar o ganho de áudio.
4. Um worker assíncrono (Cron Job embutido em Rust) que verifique periodicamente a tabela ``slots_grade``. Se um programa foi transmitido há mais de 24 horas, o arquivo físico correspondente deve ser expurgado do disco e o status alterado para 'arquivado'.

6. Plano 5: Módulo de Curadorias e Integração de Enriquecimento Automático (Spotify API)

Este módulo lida com a submissão aberta de curadorias textuais e musicais. O backend intercepta o link do Spotify inserido, extrai via expressões regulares o ID único da playlist, faz o enriquecimento de dados capturando a capa em alta resolução via API do Spotify e fornece cartões ricos para moderação em um único clique no painel administrativo.

Modelo de Meta-Prompt para a IA Arquiteta (Plano 5)

Aja como um Desenvolvedor de Integrações de API Sênior. Escreva as diretrizes para que outra IA monte o módulo de curadorias:

1. No backend Rust, crie uma rota de recepção que execute filtros de higienização Regex contra XSS armazenado em blocos textuais longos.
2. Implemente o parser de strings para URLs do Spotify, disparando um cliente HTTP (``request``) autenticado via Client Credentials do Spotify para buscar a capa oficial da playlist e gravar a URL do asset junto com o payload no Supabase.
3. No frontend SvelteKit, crie a tela administrativa de moderação contendo o feed de cards reativos. A aprovação ou rejeição deve disparar mutações em lote de clique único para alterar o status e atualizar imediatamente a visualização em www.blackmesa.page.

7. Plano 6: Dimensionamento de Storage e Infraestrutura Always Free

A rádio opera de segunda a segunda, das 06:00 às 01:00 (19 horas diárias), perfazendo um total consolidado de **133 horas semanais de transmissão**. A tabela abaixo detalha o consumo físico frente à cota de 200 GB gratuitos disponíveis no plano Oracle Always Free.

Qualidade do Áudio (Bitrate)	Consumo por Hora (MB)	Consumo Semanal (133h)	Impacto no Disco Free (200GB)
128 kbps (Padrão FM)	~56.25 MB	~7.48 GB	~3.7% do armazenamento livre
192 kbps (Alta Qualidade)	~84.37 MB	~11.22 GB	~5.6% do armazenamento livre
320 kbps (Qualidade Estúdio Máxima)	~140.62 MB	~18.70 GB	~9.3% do armazenamento livre

Decisão de Engenharia: O sistema operará obrigatoriamente na qualidade máxima de **320 kbps**. Com a rotina de limpeza cíclica programada no Plano 4, o servidor consumirá no máximo 9.3% do espaço alocado semanalmente, zerando o risco de exaustão do storage gratuito de forma perpétua.

8. Plano 7: Framework de Testes, Auditoria OWASP e CI/CD Seguro

O ciclo de desenvolvimento para o domínio `www.blackmesa.page` exige automação completa de checagem contra vulnerabilidades, testes regressivos de permissões de acesso e validação estática de dependências perigosas.

Modelo de Meta-Prompt para a IA Arquiteta (Plano 7)

Aja como um Engenheiro Líder de DevSecOps. Estruture o plano completo de esteira CI/CD e testes regressivos automatizados para outra IA programar:

- Testes de Integração em Rust (``tokio::test``) que enviem dados malformados (arquivos vazios, payloads de 300MB, executáveis injetados disfarçados de áudio) para garantir respostas HTTP 400/413 controladas.
- Testes unitários de Banco com pgTAP simulando sessões anônimas e sessões de artistas para

garantir de forma matemática que nenhuma política RLS do Supabase permita acessos cruzados (Cross-Tenant Access).

3. Um fluxo de trabalho YAML (GitHub Actions) que rode a cada Pull Request executando: ``cargo clippy`` para análise estática, ``cargo audit`` para checagem de crates vulneráveis no ecossistema Rust, `npm audit` para pacotes do SvelteKit, e impeça de forma mandatória o deploy em www.blackmesa.page caso falhas ligadas ao OWASP Top 10 sejam encontradas.